Shell Scripting

- **S**hell is a **command-line interpreter** in Linux.
- Shell scripting is writing a **set of commands in a file** to automate tasks.

Example uses:

- Automation (backup, deployment)
- System monitoring
- DevOps tasks

Types of Shells

- **Bash (Bourne Again Shell)** – most common
- Sh (Bourne Shell)
- Ksh (Korn Shell)
- Csh (C Shell)

Mostly we use: **bash**

Basic Script Structure

```
#!/bin/bash
# This is a comment

echo "Hello World"
```

Steps to run:

```
chmod +x script.sh
./script.sh
```

Variables

```
name="Tejasree"
echo $name
```

Rules:

- No spaces around =

- Use `$` to access value

User Input

```
echo "Enter your name:"
read name
echo "Hello $name"
```

Conditional Statements

if condition

```
if [ $a -gt 10 ]
then
  echo "Greater"
else
  echo "Smaller"
fi
```

Operators:

- `-eq` → equal
- `-ne` → not equal
- `-gt` → greater
- `-lt` → less

Loops

for loop

```
for i in 1 2 3
do
  echo $i
done
```

while loop

```
i=1
while [ $i -le 5 ]
do
  echo $i
  ((i++))
done
```

Functions

```
function greet() {
  echo "Hello"
}

greet
```

Arguments (Command Line Inputs)

```
echo $1
echo $2
```

Run:

```
./script.sh Teja DevOps
```

Exit Status

```
echo $?
```

`0` → success
non-zero → error

File Handling

```
if [ -f file.txt ]
then
  echo "File exists"
fi
```

File checks:

- `-f` → file exists
- `-d` → directory exists

Useful Commands in Scripts

- `echo` → print
- `read` → input

- `date` → current date
- `pwd` → present directory
- `ls` → list files

Simple Example Script

```
#!/bin/bash

echo "Enter number:"
read num

if [ $num -gt 10 ]
then
  echo "Number is greater than 10"
else
  echo "Number is less than or equal to 10"
fi
```

Real-Time Use Cases (Important for DevOps)

- Auto deployment scripts
- Backup scripts
- Log cleanup
- Server health check
- Cron jobs automation

Crontab (Scheduling)

```
crontab -e
```

Example:

```
0 2 * * * /home/user/script.sh
```

Runs daily at 2 AM